

ΠΑΝΕΠΙΣΤΗΜΙΟ ΔΥΤΙΚΗΣ ΑΤΤΙΚΗΣ
ΤΜΗΜΑ ΜΗΧΑΝΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ

ΠΤΥΧΙΑΚΗ ΕΡΓΑΣΙΑ

**Υλοποίηση διασύνδεσης υφιστάμενων
μετρητικών/καταγραφικών συστημάτων ενεργειακών
δεδομένων (ή SCADA) σε βάση δεδομένων ανοικτού
λογισμικού**

Φοιτητής: **Δημήτρης Μισούλης**

Αριθμός Μητρώου: **46136142**

Επιβλέπων Καθηγητής: **Μαϊτός Αντώνης**

Αθήνα, 2026

Περιεχόμενα

Περίληψη	3
Abstract	4
1 Εισαγωγή	5
1.1 Παρουσίαση του ερευνητικού προβλήματος	5
1.2 Σκοπός	5
1.3 Ερευνητικές υποθέσεις	6
1.4 Μηδενικές υποθέσεις	6
1.5 Οριοθετήσεις και περιορισμοί	6
1.6 Λειτουργικοί ορισμοί	6
1.7 Ανασκόπηση βιβλιογραφίας	7
1.7.1 Εμπορικά λογισμικά καταγραφικών	7
1.7.2 Βάσεις δεδομένων χρονοσειρών	7
1.7.3 Εργαλεία οπτικοποίησης	7
1.7.4 Τεχνολογίες containerization	8
1.7.5 Πρωτόκολλα επικοινωνίας SCADA	8
2 Μεθοδολογία	9
2.1 Περιγραφή της μεθοδολογίας	9
2.2 Δείγμα	9
2.3 Όργανα	10
2.4 Διαδικασία	11
2.4.1 Αρχιτεκτονική και αντιστοίχιση σε MVC	11
2.4.2 Πρόσβαση στις υπηρεσίες	12
2.4.3 Σχεδίαση της βάσης δεδομένων	12
2.4.4 Ροή εισαγωγής δεδομένων	12
2.4.5 Καθαρισμός και έλεγχος εγκυρότητας	13
2.4.6 Διαχειριστικό, ασφάλεια και αντίγραφα	14
2.4.7 Σύνδεση του Grafana και συγγραφή ερωτημάτων	14
3 Αποτελέσματα	16
3.1 Περιγραφική ανάλυση των δεδομένων	16
3.2 Επαλήθευση ακεραιότητας — αποτροπή διπλοεγγραφών	17
3.3 Κύρια ανάλυση — σενάρια οπτικοποίησης	18
3.3.1 Σενάριο 1 — Μέση θερμοκρασία ανά 5 λεπτά	18
3.3.2 Σενάριο 2 — Καταγραφή μόνο στην αλλαγή (αναφορά κατ' εξαίρεση)	20

3.3.3	Σενάριο 3 — Ημερήσια μέγιστα ηλιακής ακτινοβολίας	20
4	Συζήτηση αποτελεσμάτων	22
4.1	Επαλήθευση ερευνητικών υποθέσεων	22
4.2	Σύγκριση με εμπορικές λύσεις	22
4.3	Σύγκριση με εναλλακτικές ανοικτού κώδικα	23
4.4	Περιορισμοί και απειλές εσωτερικής εγκυρότητας	23
5	Συμπεράσματα και προτάσεις	24
5.1	Συμπεράσματα	24
5.2	Δυσκολίες που αντιμετωπίστηκαν	24
5.3	Προτάσεις για μελλοντική επέκταση	25
	Βιβλιογραφία	26
	Παράρτημα Α — Πρόσθετα ερωτήματα	27
	Παράρτημα Β — Ενδεικτικά τμήματα κώδικα	28
5.4	B.1 Ορισμός του πίνακα μετρήσεων (SQLAlchemy Model)	28
5.5	B.2 Αυτόματος εντοπισμός κεφαλίδων	28
5.6	B.3 Εισαγωγή αρχείου χωρίς διπλοεγγραφές (απλοποιημένα)	28
5.7	B.4 Έλεγχος φυσικών ορίων	29
5.8	B.5 Απόσπασμα ρύθμισης του Docker Compose	29

Περίληψη

Η παρούσα πτυχιακή εργασία παρουσιάζει τον σχεδιασμό και την υλοποίηση της εφαρμογής **MeteoLogger**, μιας ανοικτής πλατφόρμας συλλογής, αποθήκευσης και παρακολούθησης χρονοσειρών ενεργειακών και περιβαλλοντικών μετρήσεων. Το ερευνητικό πρόβλημα που αντιμετωπίζεται είναι ότι τα δεδομένα των μετρητικών/καταγραφικών οργάνων συχνά παραμένουν σε κλειστά λογισμικά και δεν αποθηκεύονται ούτε αξιοποιούνται συστηματικά. Σκοπός είναι η διασύνδεση τέτοιων συστημάτων —με πρώτη εφαρμογή έναν μετεωρολογικό σταθμό DAVIS— σε μια βάση δεδομένων ανοικτού λογισμικού.

Ως προς τη μεθοδολογία, η εφαρμογή δέχεται αρχεία μετρήσεων (Excel/CSV), τα ελέγχει και τα καθαρίζει, και τα αποθηκεύει σε βάση **PostgreSQL** με μοναδικό κλειδί τη χρονική στιγμή κάθε μέτρησης. Η διαχείριση γίνεται μέσα από διαχειριστικό γραμμένο σε **Vue** και η οπτικοποίηση με το **Grafana**, ενώ όλα εκτελούνται μέσω **Docker**. Ως δείγμα χρησιμοποιήθηκαν 106.138 πραγματικές μετρήσεις (Ιούλιος–Σεπτέμβριος 2022). Τα αποτελέσματα επιβεβαιώνουν ότι όλη η αλυσίδα λειτουργεί, ότι η χρονική στιγμή ως κλειδί αποτρέπει τις διπλοεγγραφές, και ότι τεχνικές μείωσης δεδομένων επιτρέπουν αποδοτική οπτικοποίηση (μείωση έως ~96% των σημείων). Όλα τα εργαλεία είναι ανοικτού λογισμικού.

Λέξεις-κλειδιά: χρονοσειρές, ενεργειακά δεδομένα, μετεωρολογικός σταθμός, PostgreSQL, FastAPI, Grafana, Docker, ανοικτό λογισμικό, SCADA.

Abstract

This thesis presents the design and implementation of **MeteoLogger**, an open-source platform for collecting, storing and monitoring time series of energy and environmental measurements. The research problem is that data from measurement/logging instruments often remain locked in proprietary software and are neither stored nor exploited systematically. The aim is to connect such systems —initially a DAVIS weather station— to an open-source database.

Methodologically, the application accepts measurement files (Excel/CSV), validates and cleans them, and stores them in a **PostgreSQL** database using each measurement's timestamp as a unique key. Management is performed through a **Vue** administration panel and visualization through **Grafana**, all running via **Docker**. A sample of 106,138 real measurements (July–September 2022) was used. The results confirm that the whole pipeline works, that the timestamp-as-key prevents duplicate records, and that data-reduction techniques enable efficient visualization (up to ~96% fewer points). All tools are open-source.

Keywords: time series, energy data, weather station, PostgreSQL, FastAPI, Grafana, Docker, open source, SCADA.

Κεφάλαιο 1

Εισαγωγή

1.1 Παρουσίαση του ερευνητικού προβλήματος

Στον χώρο της ενέργειας και του περιβάλλοντος, μεγάλος όγκος δεδομένων παράγεται συνεχώς από όργανα μέτρησης και καταγραφικά συστήματα: μετρητές ηλεκτρικών μεγεθών, αισθητήρες θερμοροής, μετεωρολογικοί σταθμοί και συστήματα εμποτισμού ελέγχου (SCADA). Τα συστήματα αυτά παράγουν εκτενείς χρονοσειρές μετρήσεων, οι οποίες αν αξιοποιηθούν σωστά μπορούν να αποκαλύψουν πρότυπα, τάσεις και ανωμαλίες που είναι κρίσιμα για τη βελτιστοποίηση ενεργειακών διαδικασιών, την πρόβλεψη βλαβών και τη λήψη αποφάσεων.

Στην πράξη, ωστόσο, τα δεδομένα αυτά συχνά παραμένουν «κλειδωμένα» στα ίδια τα όργανα ή σε κλειστά, εμπορικά λογισμικά, και εξάγονται χειροκίνητα σε αρχεία (π.χ. Excel) χωρίς να αποθηκεύονται συστηματικά ούτε να αξιοποιούνται για παρακολούθηση και ανάλυση. Η κατάσταση αυτή είναι ιδιαίτερα έντονη σε μικρομεσαίους οργανισμούς, ερευνητικά ιδρύματα και δημόσιους φορείς που δεν έχουν τους πόρους για ακριβή εμπορικά συστήματα διαχείρισης δεδομένων.

Το πρόβλημα αυτό δημιουργεί δύο κύριες δυσκολίες. Πρώτον, τα ιστορικά δεδομένα δεν διατηρούνται με οργανωμένο τρόπο, οπότε χάνεται η δυνατότητα μελέτης της χρονικής τους εξέλιξης και εξαγωγής χρήσιμων συμπερασμάτων. Δεύτερον, η εξάρτηση από κλειστά λογισμικά συνεπάγεται κόστος αδειών και περιορισμούς στην επεκτασιμότητα, καθιστώντας αδύνατη την ενσωμάτωση νέων πηγών δεδομένων ή τη δημιουργία προσαρμοσμένων αναλύσεων.

1.2 Σκοπός

Σκοπός της εργασίας είναι η ανάπτυξη μιας **ανοικτής πλατφόρμας** που διασυνδέει υφιστάμενα μετρητικά/καταγραφικά συστήματα με μια βάση δεδομένων ανοικτού λογισμικού, ώστε τα δεδομένα να συλλέγονται εύκολα, να αποθηκεύονται μόνιμα ως χρονοσειρές και να οπτικοποιούνται μέσω γραφημάτων, με αποκλειστική χρήση εργαλείων ανοικτού λογισμικού και εύκολη, αναπαραγώγιμη εγκατάσταση. Η πλατφόρμα αυτή στοχεύει να λύσει πρακτικό πρόβλημα που αντιμετωπίζουν καθημερινά τεχνικοί, ερευνητές και μηχανικοί που εργάζονται με καταγραφικά όργανα.

Ειδικότερα, οι επιμέρους στόχοι της εργασίας είναι:

- Σχεδίαση αρχιτεκτονικής που είναι επεκτάσιμη σε πολλαπλές πηγές δεδομένων.
- Υλοποίηση αξιόπιστης ροής εισαγωγής με αυτόματο καθαρισμό και έλεγχο ποιότητας.

- Δημιουργία διαδικτυακού διαχειριστικού με ασφαλή είσοδο και εύκολη χρήση.
- Ενσωμάτωση εργαλείου οπτικοποίησης με δυνατότητα σύνθετων ερωτημάτων.
- Εξασφάλιση εύκολης, αναπαραγωγίσιμης εγκατάστασης μέσω τεχνολογίας container.

1.3 Ερευνητικές υποθέσεις

Δεδομένου ότι πρόκειται για εργασία υλοποίησης, οι ερευνητικές υποθέσεις διατυπώνονται ως προτάσεις που ελέγχονται από το ίδιο το αποτέλεσμα της κατασκευής:

- **Y1:** Είναι εφικτή η υλοποίηση, με εργαλεία αποκλειστικά ανοικτού λογισμικού, μιας πλατφόρμας που συλλέγει, αποθηκεύει και οπτικοποιεί χρονοσειρές μετρήσεων.
- **Y2:** Η χρήση της χρονικής στιγμής ως μοναδικού κλειδιού εξασφαλίζει την ακεραιότητα των δεδομένων, αποτρέποντας τις διπλοεγγραφές.
- **Y3:** Τεχνικές μείωσης δεδομένων (μέση τιμή ανά χρονικό διάστημα, καταγραφή μόνο στην αλλαγή) επιτρέπουν την αποδοτική οπτικοποίηση μεγάλου όγκου μετρήσεων.

1.4 Μηδενικές υποθέσεις

- **Ho-1:** Δεν είναι εφικτή η υλοποίηση τέτοιας πλατφόρμας μόνο με ανοικτό λογισμικό.
- **Ho-2:** Η χρονική στιγμή ως κλειδί δεν αποτρέπει τις διπλοεγγραφές.
- **Ho-3:** Οι τεχνικές μείωσης δεδομένων δεν βελτιώνουν την οπτικοποίηση του όγκου.

Ο έλεγχος των παραπάνω γίνεται στα κεφάλαια Αποτελέσματα και Συζήτηση.

1.5 Οριοθετήσεις και περιορισμοί

- Η υλοποίηση αφορά **μία πηγή δεδομένων** (μετεωρολογικός σταθμός DAVIS)· τα υπόλοιπα όργανα υποστηρίζονται σχεδιαστικά αλλά όχι στην παρούσα φάση.
- Η εισαγωγή γίνεται **από αρχεία** (Excel/CSV) και όχι με άμεση επικοινωνία με τα όργανα σε πραγματικό χρόνο.
- Το **δείγμα** δεδομένων καλύπτει την καλοκαιρινή περίοδο (Ιούλιος–Σεπτέμβριος 2022), οπότε ακραία φαινόμενα ψύχους ή θυελλώδους ανέμου δεν εμφανίζονται.
- Η εγκατάσταση δοκιμάστηκε σε **τοπικό δίκτυο** (Ubuntu Server), χωρίς δημόσια πρόσβαση από το διαδίκτυο.
- Δεν υλοποιήθηκε μηχανισμός **αυτόματης λήψης** δεδομένων σε πραγματικό χρόνο· αυτό αποτελεί πρόταση για μελλοντική επέκταση.

1.6 Λειτουργικοί ορισμοί

- **Χρονοσειρά:** ακολουθία μετρήσεων ενός μεγέθους σε διαδοχικές χρονικές στιγμές, με σταθερό ή μεταβαλλόμενο χρονικό βήμα.
- **Μέτρηση:** ένα σύνολο τιμών (θερμοκρασία, υγρασία κ.λπ.) που αντιστοιχεί σε μία χρονική στιγμή.

- **Χρονική σφραγίδα (timestamp):** η ημερομηνία και ώρα μιας μέτρησης, που χρησιμοποιείται ως μοναδικό αναγνωριστικό.
- **Διπλοεγγραφή:** δύο εγγραφές με την ίδια χρονική στιγμή· δεν επιτρέπεται στο σύστημα.
- **Ανοικτό λογισμικό:** λογισμικό με ελεύθερα διαθέσιμο πηγαίο κώδικα, που επιτρέπει χρήση, τροποποίηση και αναδιανομή.
- **Container:** απομονωμένο περιβάλλον εκτέλεσης ενός τμήματος της εφαρμογής μέσω Docker, που εξασφαλίζει ίδια συμπεριφορά σε κάθε μηχανήμα.
- **Καταγραφή κατ' εξαίρεση (exception reporting):** διατήρηση νέας τιμής μόνο όταν το μέγεθος μεταβληθεί πέρα από ένα όριο· τεχνική που προέρχεται από τα συστήματα SCADA.
- **Αντίστροφος διαμεσολαβητής (reverse proxy):** εξυπηρετητής που δέχεται αιτήματα HTTP και τα προωθεί στην κατάλληλη εσωτερική υπηρεσία βάσει κανόνων δρομολόγησης.

1.7 Ανασκόπηση βιβλιογραφίας

1.7.1 Εμπορικά λογισμικά καταγραφικών

Τα εμπορικά λογισμικά καταγραφικών (π.χ. το **WeatherLink** της DAVIS, το **WEBfactory** για SCADA, το **OSIsoft PI System**) προσφέρουν ολοκληρωμένη λύση οπτικοποίησης και διαχείρισης, αλλά παρουσιάζουν σημαντικούς περιορισμούς: είναι κλειστά συστήματα που περιορίζουν την πρόσβαση στα «ωμά» δεδομένα, απαιτούν δαπανηρές άδειες χρήσης και δεν επιτρέπουν εύκολη επέκταση ή ενσωμάτωση με άλλα συστήματα. Επιπλέον, η εξαγωγή ιστορικών δεδομένων γίνεται συνήθως σε αρχεία Excel ή CSV, τα οποία στη συνέχεια δεν αξιοποιούνται περαιτέρω.

1.7.2 Βάσεις δεδομένων χρονοσειρών

Στον χώρο του ανοικτού λογισμικού υπάρχουν αρκετές επιλογές για αποθήκευση χρονοσειρών:

Η **PostgreSQL** αποτελεί ώριμη, αξιόπιστη σχεσιακή βάση δεδομένων κατάλληλη για επιστημονικά δεδομένα και χρονοσειρές. Υποστηρίζει ευρετήρια σε στήλες τύπου timestamp, διαθέτει πλούσιες συναρτήσεις χρονικής ομαδοποίησης και συνδέεται άμεσα με εργαλεία οπτικοποίησης όπως το Grafana. Το σχεσιακό μοντέλο της επιτρέπει εύκολη επέκταση με πίνακες συσκευών, χρηστών και μεταδεδομένων.

Η **InfluxDB** είναι εξειδικευμένη βάση χρονοσειρών που προσφέρει υψηλές επιδόσεις για εγγραφή και ανάγνωση σε πραγματικό χρόνο. Ωστόσο, εισάγει τη δική της γλώσσα ερωτημάτων (Flux ή InfluxQL), η οποία απαιτεί επιπλέον μάθηση, ενώ η έκδοση 2.x έχει αλλάξει σημαντικά το μοντέλο δεδομένων σε σχέση με προηγούμενες εκδόσεις, δημιουργώντας ζητήματα συμβατότητας.

Το **TimescaleDB** είναι επέκταση της PostgreSQL που προσθέτει βελτιστοποιήσεις για χρονοσειρές (hypertables, συνεχείς συναθροίσεις). Αποτελεί ενδιαφέρουσα επιλογή αλλά εισάγει επιπλέον εξάρτηση και πολυπλοκότητα εγκατάστασης.

1.7.3 Εργαλεία οπτικοποίησης

Το **Grafana** έχει καθιερωθεί ως το de facto εργαλείο οπτικοποίησης χρονοσειρών ανοικτού λογισμικού. Συνδέεται άμεσα με PostgreSQL, InfluxDB, Prometheus και δεκάδες άλλες πηγές δεδομένων, επιτρέπει τη δη-

μιουργία διαδραστικών πινάκων ελέγχου (dashboards) και υποστηρίζει ειδοποιήσεις (alerting) όταν μια τιμή ξεπεράσει κάποιο όριο.

Το **Prometheus** είναι σύστημα παρακολούθησης και συλλογής μετρικών που «ρωτάει» (scrape) ζωντανά τα συστήματα σε τακτά χρονικά διαστήματα. Προορίζεται κυρίως για παρακολούθηση της υγείας υποδομών (servers, containers, εφαρμογές) σε πραγματικό χρόνο. Δεν είναι κατάλληλο για εισαγωγή ιστορικών αρχείων σε σχεσιακή βάση.

1.7.4 Τεχνολογίες containerization

Το **Docker** αποτελεί σήμερα τον κυρίαρχο τρόπο συσκευασίας και διανομής εφαρμογών. Κάθε υπηρεσία εκτελείται σε απομονωμένο container με όλες τις εξαρτήσεις της, εξασφαλίζοντας ίδια συμπεριφορά ανεξάρτητα από το λειτουργικό σύστημα του κεντρικού μηχανήματος. Το **Docker Compose** επιτρέπει τον ορισμό και τη διαχείριση πολλαπλών containers ως ενιαία εφαρμογή μέσω ενός αρχείου YAML.

1.7.5 Πρωτόκολλα επικοινωνίας SCADA

Τα συστήματα SCADA χρησιμοποιούν εξειδικευμένα πρωτόκολλα για την επικοινωνία με τα όργανα μέτρησης. Το **Modbus TCP/RTU** είναι το πλέον διαδεδομένο πρωτόκολλο στη βιομηχανία, με δεκαετίες ιστορίας και ευρεία υποστήριξη από κατασκευαστές. Τα **IEC 60870-5-104** και **IEC 61850** χρησιμοποιούνται κυρίως στον ηλεκτρικό τομέα. Στην παρούσα εργασία η επικοινωνία γίνεται μέσω αρχείων (offline), αλλά η αρχιτεκτονική έχει σχεδιαστεί ώστε να επιτρέπει μελλοντική ενσωμάτωση αυτών των πρωτοκόλλων.

Κεφάλαιο 2

Μεθοδολογία

2.1 Περιγραφή της μεθοδολογίας

Η προσέγγιση που ακολουθήθηκε είναι κατασκευαστική (design-and-build): σχεδιάστηκε και υλοποιήθηκε ένα πλήρες σύστημα λογισμικού, το οποίο στη συνέχεια δοκιμάστηκε με πραγματικά δεδομένα. Η επιλογή αυτής της μεθοδολογίας κρίθηκε κατάλληλη γιατί ο στόχος είναι η δημιουργία λειτουργικού προϊόντος και όχι η θεωρητική ανάλυση ή η σύγκριση υπαρχόντων λύσεων. Τα βήματα ήταν:

1. **Ανάλυση απαιτήσεων:** καθορισμός των λειτουργικών και μη λειτουργικών απαιτήσεων της πλατφόρμας.
2. **Επιλογή τεχνολογιών:** αξιολόγηση και επιλογή εργαλείων ανοικτού λογισμικού βάσει κριτηρίων ωριμότητας, κοινότητας και καταλληλότητας.
3. **Σχεδίαση αρχιτεκτονικής:** ορισμός των components και των σχέσεων μεταξύ τους.
4. **Σχεδίαση βάσης δεδομένων:** ορισμός του σχήματος δεδομένων και των ευρετηρίων.
5. **Υλοποίηση backend:** ανάπτυξη του REST API σε Python/FastAPI.
6. **Υλοποίηση frontend:** ανάπτυξη του διαχειριστικού σε Vue.
7. **Ρύθμιση υποδομής:** διαμόρφωση Docker Compose, Nginx reverse proxy και backup.
8. **Δοκιμή με πραγματικά δεδομένα:** εισαγωγή και επαλήθευση με 106.138 μετρήσεις.

2.2 Δείγμα

Ως δείγμα χρησιμοποιήθηκαν πραγματικές μετρήσεις ενός μετεωρολογικού σταθμού **DAVIS Vantage Pro2**, για την περίοδο **10/7/2022 έως 30/9/2022**, με ρυθμό **μίας μέτρησης ανά λεπτό**. Το σύνολο περιλαμβάνει **106.138 μετρήσεις**, που αντιστοιχούν σε περίπου 73,7 ημέρες συνεχούς λειτουργίας. Η εξαγωγή των δεδομένων έγινε μέσω του λογισμικού WeatherLink της DAVIS σε αρχείο Excel.

Κάθε μέτρηση περιέχει τα ακόλουθα μεγέθη:

- **Θερμοκρασία αέρα** (°C): μετρούμενη σε σκιά, σε ύψος 1,5 m.
- **Σχετική υγρασία** (%): αναλογία υδρατμών στον αέρα.
- **Βαρομετρική πίεση** (hPa): ανηγμένη στο επίπεδο της θάλασσας.
- **Ταχύτητα ανέμου** (km/h): μέση τιμή ανά λεπτό.
- **Διεύθυνση ανέμου** (μοίρες): 0–360°.
- **Ριπή ανέμου** (km/h) και **διεύθυνση ριπής**: μέγιστη ταχύτητα εντός του λεπτού.

- **Βροχόπτωση** (mm): συσσωρευτική εντός του λεπτού.
- **Ρυθμός βροχής** (mm/h): στιγμιαίος ρυθμός.
- **Σημείο δρόσου** (°C): θερμοκρασία στην οποία ο αέρας κορεννύεται.
- **Αισθητή θερμοκρασία / Wind Chill** (°C): εκτιμώμενη θερμική αίσθηση.
- **Ηλιακή ακτινοβολία** (W/m²): ολική εισερχόμενη ηλιακή ακτινοβολία.
- **Δείκτης UV**: ένταση υπεριώδους ακτινοβολίας σε αδιάστατη κλίμακα.

2.3 Όργανα

Ως «όργανα» της εργασίας νοούνται τόσο το μετρητικό όργανο όσο και τα εργαλεία λογισμικού. Κοινό χαρακτηριστικό των εργαλείων λογισμικού είναι ότι όλα είναι **ανοικτού λογισμικού**, εξασφαλίζοντας μηδενικό κόστος αδειών και ελευθερία επέκτασης.

Μετεωρολογικός σταθμός DAVIS Vantage Pro2. Το όργανο που παράγει τα δεδομένα. Πρόκειται για επαγγελματικό μετεωρολογικό σταθμό με αισθητήρες θερμοκρασίας, υγρασίας, βαρόμετρο, ανεμόμετρο, βροχόμετρο, πυρανόμετρο (ηλιακή ακτινοβολία) και αισθητήρα UV. Εξάγει τις μετρήσεις μέσω του λογισμικού WeatherLink σε αρχεία Excel.

Python 3.11 και FastAPI. Η λογική της εφαρμογής υλοποιήθηκε σε Python με το πλαίσιο FastAPI. Η Python επιλέχθηκε για το ώριμο οικοσύστημα επεξεργασίας δεδομένων, ιδίως τη βιβλιοθήκη **pandas** για ανάγνωση Excel/CSV και επεξεργασία πινάκων δεδομένων. Η **SQLAlchemy** χρησιμοποιήθηκε ως ORM για την επικοινωνία με τη βάση. Το FastAPI είναι σύγχρονο, γρήγορο, παράγει αυτόματη τεκμηρίωση (Swagger UI) και ενσωματώνεται εύκολα σε Docker.

PostgreSQL 15. Επιλέχθηκε ως βάση γιατί είναι ώριμη, αξιόπιστη, κατάλληλη για επιστημονικά δεδομένα και χρονοσειρές (ευρετήριο B-tree στη χρονική σφραγίδα, συναρτήσεις `date_trunc`, `generate_series`, `window functions`), και συνδέεται **άμεσα** με το Grafana μέσω ενσωματωμένου plugin. Έναντι της InfluxDB προτιμήθηκε γιατί διατηρεί το σχεσιακό μοντέλο (χρήσιμο για μελλοντικούς πίνακες συσκευών/μεταδεδομένων) χωρίς εξάρτηση από εξειδικευμένη γλώσσα ερωτημάτων.

Grafana 11.4.0 (και γιατί όχι Prometheus). Η οπτικοποίηση γίνεται με το Grafana, που διαβάζει απευθείας από την PostgreSQL και εκτελεί ερωτήματα SQL. Επιτρέπει τη δημιουργία πινάκων ελέγχου (dashboards) με πολλαπλά γραφήματα, υποστηρίζει μεταβλητές χρόνου (`$__timeFilter`, `$__timeGroupAlias`) και μπορεί να ενσωματωθεί σε εξωτερικές σελίδες μέσω `iframe`. Το Prometheus, αντίθετα, είναι σύστημα που «ρωτάει» ζωντανά τα συστήματα (scraping) και προορίζεται για παρακολούθηση υποδομών σε πραγματικό χρόνο, όχι για εισαγωγή ιστορικών αρχείων σε σχεσιακή βάση· γι' αυτό δεν επιλέχθηκε (αναφέρεται όμως στις προτάσεις για μελλοντική παρακολούθηση της υποδομής).

Vue 3 και ElementUI Plus. Το διαχειριστικό υλοποιήθηκε με το Vue 3 (Composition API) και τη βιβλιοθήκη έτοιμων στοιχείων ElementUI Plus, για γρήγορη ανάπτυξη καθαρής, αντιδραστικής διεπαφής. Η επικοινωνία με το backend γίνεται μέσω αξόνων HTTP (axios).

Docker 24 και Docker Compose. Όλα εκτελούνται σε Docker, το οποίο «πακετάρει» κάθε τμήμα σε απομονωμένο container με τις εξαρτήσεις του, εξασφαλίζοντας ίδια συμπεριφορά σε κάθε μηχανήμα. Το Docker Compose επιτρέπει τον ορισμό όλων των services σε ένα αρχείο `docker-compose.yml` και την εκκίνηση ολόκληρης της εφαρμογής με μία εντολή (`docker compose up -d`).

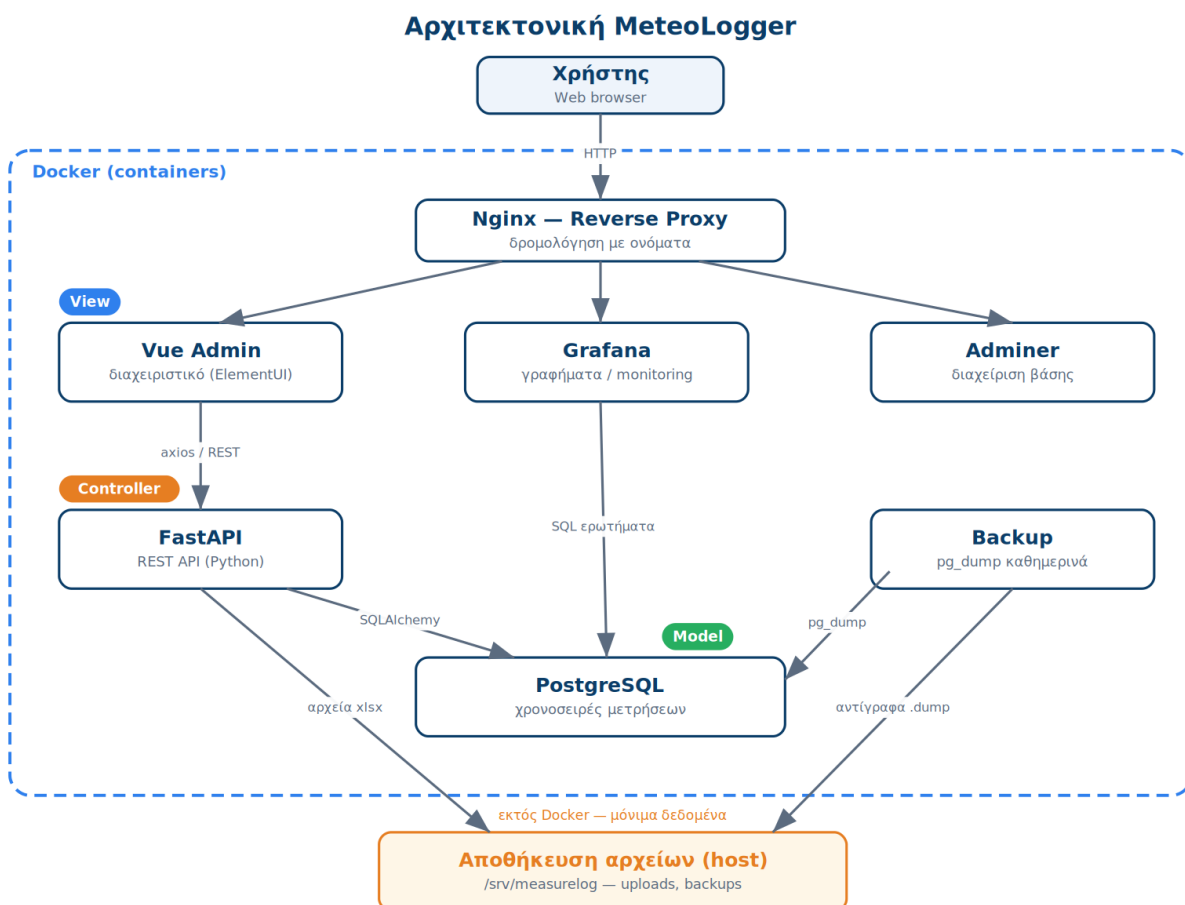
Nginx 1.27. Χρησιμοποιείται ως αντίστροφος διαμεσολαβητής (reverse proxy) που δρομολογεί τα εισερχόμενα HTTP αιτήματα στην κατάλληλη υπηρεσία βάσει του ονόματος του κεντρικού υπολογιστή (virtual host). Επιτρέπει την πρόσβαση σε όλες τις υπηρεσίες μέσω της ίδιας θύρας (80).

Adminer. Ελαφρύ web-based εργαλείο διαχείρισης βάσεων δεδομένων, κατάλληλο για άμεση επιθεώρηση των δεδομένων κατά την ανάπτυξη και τον εντοπισμό σφαλμάτων.

2.4 Διαδικασία

2.4.1 Αρχιτεκτονική και αντιστοίχιση σε MVC

Η αρχιτεκτονική φαίνεται στην Εικόνα 1. Ο χρήστης αλληλεπιδρά μέσω φυλλομετρητή. Ένας **αντίστροφος διαμεσολαβητής (Nginx)** δρομολογεί τα αιτήματα με βάση το virtual hostname προς το διαχειριστικό (Vue), το Grafana ή το Adminer. Το διαχειριστικό επικοινωνεί με το **FastAPI**, το οποίο αποθηκεύει τα δεδομένα στην **PostgreSQL** και τα αρχεία στον δίσκο του εξυπηρετητή. Το Grafana διαβάζει απευθείας από την PostgreSQL, ενώ ξεχωριστή υπηρεσία δημιουργεί καθημερινά αντίγραφα ασφαλείας.



Εικόνα 2.1: Αρχιτεκτονική της εφαρμογής MeteoLogger.

Η οργάνωση ακολουθεί το πρότυπο **MVC (Model-View-Controller)**:

- **Model:** η PostgreSQL με τα αντικείμενα δεδομένων (πίνακας `weather_data`) και τα SQLAlchemy models στο backend.
- **View:** το διαχειριστικό (Vue) για τη διαχείριση δεδομένων και τα γραφήματα (Grafana) για την οπτικοποίηση.
- **Controller:** το FastAPI που συντονίζει τη λογική, επεξεργάζεται τα αιτήματα και ενορχηστρώνει τη ροή δεδομένων μεταξύ View και Model.

2.4.2 Πρόσβαση στις υπηρεσίες

Κάθε τμήμα εκτελείται ως ξεχωριστό container και είναι προσβάσιμο είτε με ονοματισμένη διεύθυνση (μέσω του reverse proxy, θύρα 80) είτε, κατά την ανάπτυξη, απευθείας με `http://<IP>:8000/docs`. Για να λειτουργεί η ονοματισμένη πρόσβαση, προστίθενται στο αρχείο `hosts` του λειτουργικού συστήματος οι αντιστοιχίσεις (π.χ. `192.168.1.x admin.meteorologger.local`).

Υπηρεσία (container)	Ρόλος	Θύρα	Ονοματισμένη πρόσβαση	Απευθείας (dev)
frontend (Vue)	Διαχειριστικό	80 (proxy)	<code>http://admin.meteorologger.local</code>	
api (FastAPI)	REST API / Swagger	8000	(μέσω <code>http://<IP>:8000/docs</code> διαχειριστικού)	
grafana	Γραφήματα	3000	<code>http://grafana.meteorologger.local</code>	
adminer	Διαχείριση βάσης	8080	<code>http://adminer.meteorologger.local</code>	
db (PostgreSQL)	Βάση δεδομένων	5432	— (εσωτερικό)	<code><IP>:5432</code>
proxy (Nginx)	Reverse proxy	80	(δρομολογεί)	—
backup	Αντίγραφα ασφαλείας	—	(παρασκήνιο)	—

2.4.3 Σχεδίαση της βάσης δεδομένων

Οι μετρήσεις αποθηκεύονται σε έναν κεντρικό πίνακα, τον `weather_data`. Η σχεδίαση ακολουθεί την αρχή της απλότητας: ένας πίνακας με επαρκή αριθμό στηλών για όλα τα μεγέθη του σταθμού DAVIS, με δυνατότητα μελλοντικής επέκτασης.

Κάθε γραμμή αντιστοιχεί σε μία χρονική στιγμή. Η στήλη `timestamp` ορίστηκε ως **μοναδική (UNIQUE)**, υλοποιώντας σε επίπεδο βάσης την αρχή ότι δεν επιτρέπονται διπλοεγγραφές — ανεξάρτητα από το αν η εφαρμογή ελέγχει πρώτα. Επίσης, δημιουργήθηκε **B-tree ευρετήριο (INDEX)** στη στήλη `timestamp`, εξασφαλίζοντας γρήγορα χρονικά ερωτήματα ακόμη και με εκατοντάδες χιλιάδες γραμμές.

Οι βασικές στήλες είναι: `id` (αυτόματα αυξανόμενο αναγνωριστικό), `timestamp` (μοναδική χρονική σφραγίδα), `temp`, `humidity`, `barometer`, `pressure`, `windspeed`, `winddirection`, `windgust`, `windgustdir`, `rain`, `rainrate`, `dewpoint`, `windchill`, `radiation`, `uv`. Όλες οι μετρικές στήλες είναι τύπου `FLOAT` και επιτρέπουν `NULL`, ώστε μια μη έγκυρη τιμή να μηδενίζεται χωρίς να χάνεται ολόκληρη η γραμμή.

2.4.4 Ροή εισαγωγής δεδομένων

Η ροή εισαγωγής είναι η κεντρική λειτουργία της εφαρμογής και περιλαμβάνει τα εξής βήματα:

1. Ο χρήστης ανεβάζει αρχείο Excel ή CSV από το διαχειριστικό μέσω φόρμας multipart/form-data.
2. Το FastAPI λαμβάνει το αρχείο και το αποθηκεύει **αυτούσιο** σε φάκελο του εξυπηρετητή (για μελλοντική λήψη και επαλήθευση).
3. Το αρχείο διαβάζεται με pandas. Λόγω του ότι τα αρχεία WeatherLink περιέχουν άσχετες γραμμές στην αρχή (τίτλος σταθμού, πληροφορίες εξαγωγής), εφαρμόζεται **αυτόματος εντοπισμός** της γραμμής κεφαλίδων: σαρώνονται οι πρώτες γραμμές ώσπου να βρεθεί αυτή που περιέχει τα αναμενόμενα ονόματα στηλών.
4. Δημιουργείται η χρονική σφραγίδα (timestamp) από τον συνδυασμό των στηλών ημερομηνίας και ώρας, με μετατροπή στον κατάλληλο τύπο datetime.
5. Οι τιμές **καθαρίζονται** (αφαίρεση κενών, μετατροπή σε αριθμούς) και **ελέγχονται** ως προς τα φυσικά τους όρια.
6. Αναζητούνται οι ήδη υπάρχουσες χρονικές σφραγίδες στη βάση και αφαιρούνται από το σύνολο προς εισαγωγή.
7. Οι νέες, έγκυρες μετρήσεις **εισάγονται ομαδικά** (bulk insert) στη βάση για αποδοτικότητα.
8. Επιστρέφεται αναφορά με τον αριθμό εγγραφών που εισήχθησαν, παραλείφθηκαν και απορρίφθηκαν.

2.4.5 Καθαρισμός και έλεγχος εγκυρότητας

Στα πραγματικά δεδομένα διαπιστώθηκε ότι, σε μικρό ποσοστό γραμμών, οι στήλες ήταν μετατοπισμένες λόγω σφάλματος του λογισμικού εξαγωγής, με αποτέλεσμα φυσικά αδύνατες τιμές (π.χ. υγρασία 1200%, θερμοκρασία -999°C). Για την αντιμετώπιση αυτού, ορίστηκαν **φυσικά όρια** ανά μέγεθος και όσες τιμές βρέθηκαν εκτός ορίων μηδενίζονται (NULL), **διατηρώντας τις υπόλοιπες τιμές της γραμμής** αναλλοίωτες:

```
BOUNDS = {
    "temp": (-40, 60),    "humidity": (0, 1),
    "barometer": (800, 1100), "windspeed": (0, 60),
    "dewpoint": (-40, 40), "windchill": (-60, 60),
    "radiation": (0, 1500), "uv": (0, 20),
}

def validate_ranges(df):
    for col, (lo, hi) in BOUNDS.items():
        if col in df.columns:
            df[col] = df[col].where(df[col].between(lo, hi)) # -> NULL
    bad = df["temp"].notna() & (df["dewpoint"] > df["temp"] + 0.5)
    df.loc[bad, "dewpoint"] = None #
    return df
```

Εφαρμόστηκε επίσης ένας φυσικός έλεγχος διασταύρωσης: το **σημείο δρόσου** δεν μπορεί να υπερβαίνει τη θερμοκρασία αέρα (αν συμβαίνει, η τιμή είναι εσφαλμένη). Η στρατηγική «NULL αντί απόρριψης» επιλέχθηκε συνειδητά: μια γραμμή με μία κακή τιμή σε ένα μέγεθος εξακολουθεί να περιέχει έγκυρες τιμές για τα υπόλοιπα.

2.4.6 Διαχειριστικό, ασφάλεια και αντίγραφα

Το διαχειριστικό (Vue) προσφέρει πλήρη σειρά λειτουργιών: είσοδο με κωδικό, επισκόπηση με στατιστικά (σύνολο μετρήσεων, εύρος ημερομηνιών, τελευταία εγγραφή), εισαγωγή Excel και CSV, λίστα/λήψη αποθηκευμένων αρχείων, ενεργοποίηση αντιγράφων ασφαλείας και ενσωματωμένα γραφήματα Grafana μέσω iframe.

Η ασφάλεια υλοποιείται με **JWT tokens**: η είσοδος με σωστά διαπιστευτήρια εκδίδει ένα βραχύχρονο token (προεπιλεγμένη διάρκεια 60 λεπτά) που αποθηκεύεται στο localStorage του φυλλομετρητή και συνοδεύει κάθε αίτημα ως Authorization header. Αιτήματα χωρίς έγκυρο token απορρίπτονται από το FastAPI.

Μια ξεχωριστή υπηρεσία (container backup) εκτελεί καθημερινά στις 02:00 την εντολή pg_dump και αποθηκεύει το αποτέλεσμα σε συμπιεσμένο αρχείο. Διατηρούνται τα **επτά πιο πρόσφατα** αντίγραφα, ώστε να καλύπτεται μια εβδομάδα ιστορικού σε περίπτωση αποτυχίας.

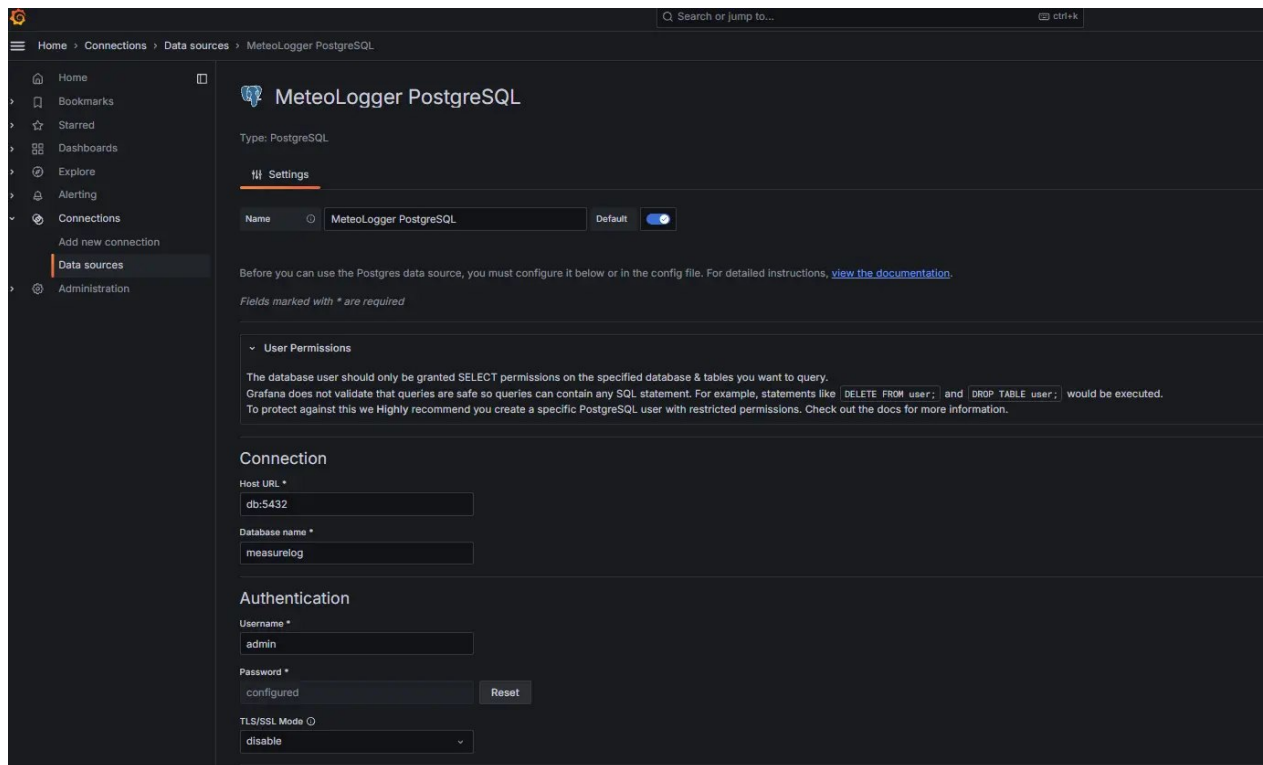
2.4.7 Σύνδεση του Grafana και συγγραφή ερωτημάτων

Το Grafana συνδέεται στην PostgreSQL με τις κατάλληλες ρυθμίσεις (εξυπηρετητής db:5432, βάση measurelog, χρήστης/κωδικός), όπως στην Εικόνα 2. Ο συντάκτης ερωτημάτων προσφέρει οπτικό επιλογέα (Builder) για απλές περιπτώσεις και λειτουργία **Code** για απευθείας σύνταξη SQL. Στην παρούσα εργασία επιλέχθηκε αποκλειστικά η **SQL** γιατί:

- Τα σενάρια απαιτούν λογική που ο οπτικός επιλογέας δεν εκφράζει (window functions, φιλτράρισμα, σύγκριση με προηγούμενη τιμή).
- Η SQL είναι ακριβής, επαναλήψιμη και αυτο-τεκμηριωμένη.
- Διευκολύνει τη μελλοντική συντήρηση και τροποποίηση των ερωτημάτων.

Το Grafana παρέχει ειδικές μακροεντολές για χρονικά ερωτήματα:

- `$__timeFilter(timestamp)`: προσθέτει αυτόματα συνθήκη WHERE με το εύρος χρόνου που έχει επιλέξει ο χρήστης στο dashboard.
- `$__timeGroupAlias(timestamp, '5m')`: ομαδοποιεί τα δεδομένα σε διαστήματα (εδώ 5 λεπτών) και επιστρέφει τη στήλη με κατάλληλο alias για το Grafana.



Εικόνα 2.2: Ρυθμίσεις σύνδεσης του Grafana με τη βάση PostgreSQL.

Κεφάλαιο 3

Αποτελέσματα

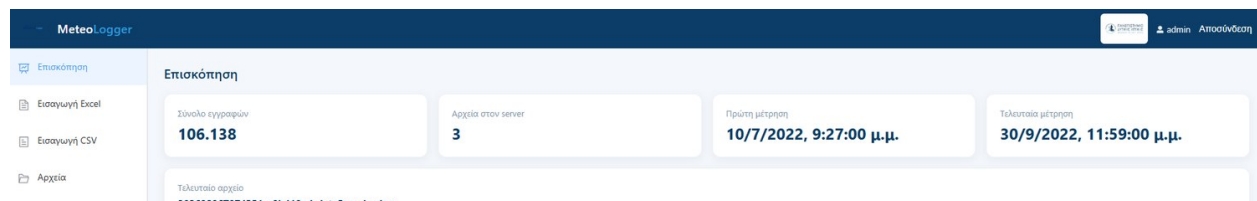
3.1 Περιγραφική ανάλυση των δεδομένων

Μετά την εισαγωγή, αποθηκεύτηκαν και οι 106.138 μετρήσεις χωρίς απώλειες. Η επισκόπηση του διαχειριστικού (Εικόνα 3) επιβεβαιώνει το πλήθος και το εύρος των ημερομηνιών (10/7/2022–30/9/2022). Ο Πίνακας 1 συνοψίζει τα βασικά στατιστικά των κύριων μεγεθών.

Πίνακας 3.1: Περιγραφικά στατιστικά των κύριων μεγεθών (δείγμα 106.138 μετρήσεων).

Μέγεθος	Ελάχιστο	Διάμεσος	Μέσος	Μέγιστο
Θερμοκρασία (°C)	14,6	28,1	27,9	38,1
Υγρασία (%)	15	40	50	89
Βαρομ. πίεση (hPa)	1001	1011	1012	1022
Ταχύτητα ανέμου (km/h)	0	6,4	7,2	45,0
Ρυθμός βροχής (mm/h)	0	0	2,1	65,0
Ηλιακή ακτινοβολία (W/m ²)	0	102	198	1.021
Δείκτης UV	0	2	3,1	11

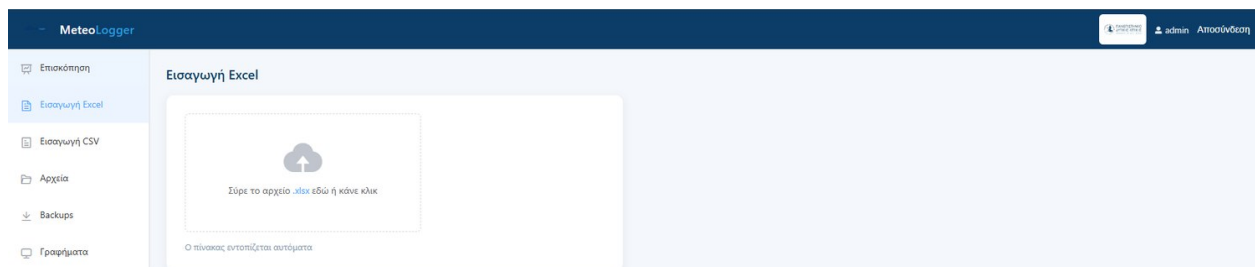
Τα στατιστικά αντικατοπτρίζουν τα χαρακτηριστικά της καλοκαιρινής Μεσογειακής περιόδου: υψηλές θερμοκρασίες με αιχμές κοντά στους 38°C, σχετικά χαμηλή υγρασία (διάμεσος 40%), μέτριοι άνεμοι και πολύ υψηλές τιμές ηλιακής ακτινοβολίας (μέγιστο 1.021 W/m²). Η μηδενική διάμεσος βροχόπτωσης επιβεβαιώνει ότι η βροχή είναι σπάνιο φαινόμενο εντός του δείγματος.



Εικόνα 3.1: Επισκόπηση του διαχειριστικού με τα βασικά στατιστικά.

Η εισαγωγή αρχείου Excel και η λίστα των αποθηκευμένων αρχείων φαίνονται στις Εικόνες 4 και 5.

Η επαλήθευση ότι τα δεδομένα αποθηκεύτηκαν στη βάση έγινε μέσω του Adminer: σύνδεση στη βάση (Εικόνα 6) και προβολή του πίνακα weather_data (Εικόνα 7). Οι κενές (NULL) τιμές που εμφανίζονται σε

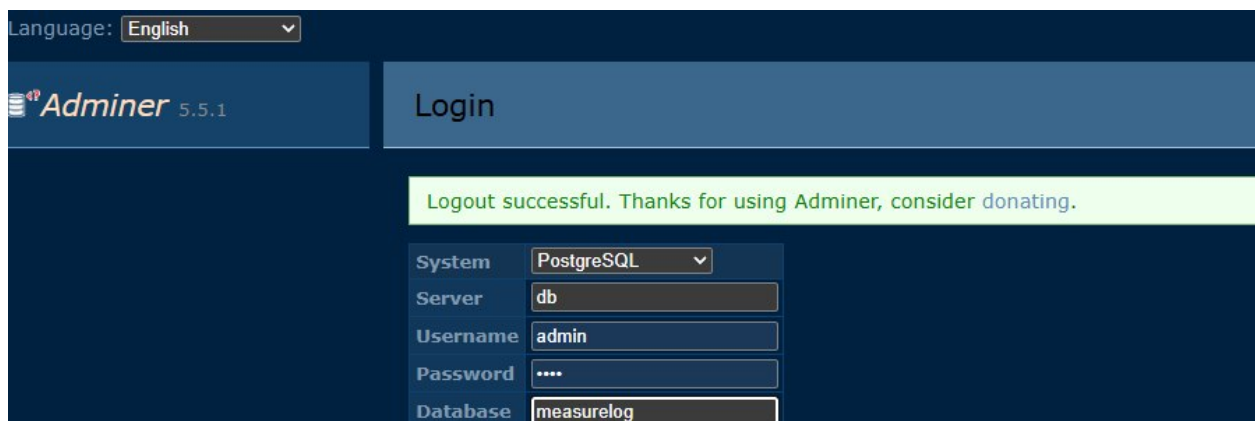


Εικόνα 3.2: Εισαγωγή αρχείου Excel από το διαχειριστικό.



Εικόνα 3.3: Λίστα αποθηκευμένων αρχείων με δυνατότητα λήψης.

ορισμένες γραμμές είναι ακριβώς όσες μηδένισε ο έλεγχος εγκυρότητας, γεγονός που επιβεβαιώνει τη σωστή λειτουργία του καθαρισμού.



Εικόνα 3.4: Σύνδεση στη βάση μέσω του Adminer.

3.2 Επαλήθευση ακεραιότητας — αποτροπή διπλοεγγραφών

Για την επαλήθευση της Y2 εκτελέστηκε η εξής δοκιμή: το ίδιο αρχείο Excel εισήχθη **δεύτερη φορά** στο σύστημα. Το αποτέλεσμα ήταν μηδενικές νέες εγγραφές — το σύστημα εντόπισε ότι όλες οι χρονικές σφραγίδες του αρχείου υπήρχαν ήδη στη βάση και τις παρέλειψε. Ο αριθμός γραμμών στον πίνακα `weather_data` παρέμεινε αμετάβλητος στις 106.138.

Αυτή η συμπεριφορά επιτυγχάνεται σε δύο επίπεδα:

1. **Επίπεδο εφαρμογής:** το FastAPI αναζητά τις υπάρχουσες χρονικές σφραγίδες πριν την εισαγωγή και αφαιρεί τις διπλές από το σύνολο προς επεξεργασία.

	id	timestamp	temp	humidity	barometer	pressure	windspeed	winddirection	windgust	windgustdir	rain	rainrate	dewpoint	windchill	radiation	uv	
edit	1	2022-09-30 23:59:00	23.72	0.52	1019.27	30.01	0	1.6	23	0	0	13.27	23.72	0	0	NULL	
edit	2	2022-09-30 23:58:00	23.78	0.52	1019.3	30.01	0	1.6	23	0	0	13.32	23.78	0	0	NULL	
edit	3	2022-09-30 23:57:00	23.72	0.52	1019.37	30.01	1.6	22.5	3.2	23	0	0	13.27	23.72	0	0	
edit	4	2022-09-30 23:56:00	23.78	0.52	1019.4	30.01	1.6	22.5	1.6	23	0	0	13.32	23.78	0	0	
edit	5	2022-09-30 23:55:00	23.78	0.52	1019.3	30.01	1.6	22.5	3.2	23	0	0	13.32	23.78	0	0	
edit	6	2022-09-30 23:54:00	23.78	0.52	1019.34	30.01	1.6	22.5	3.2	23	0	0	13.32	23.78	0	0	
edit	7	2022-09-30 23:53:00	23.78	0.52	1019.3	30.01	1.6	45	3.2	45	0	0	13.32	23.78	0	0	
edit	8	2022-09-30 23:52:00	23.83	0.52	1019.24	30.01	1.6	45	3.2	45	0	0	13.37	23.83	0	0	
edit	9	2022-09-30 23:51:00	23.83	0.52	1019.27	30.01	1.6	67.5	3.2	68	0	0	13.37	23.83	0	0	
edit	10	2022-09-30 23:50:00	23.83	0.52	1019.27	30.01	1.6	67.5	3.2	68	0	0	13.37	23.83	0	0	
edit	11	2022-09-30 23:49:00	23.83	0.52	1019.27	30.01	1.6	67.5	3.2	68	0	0	13.37	23.83	0	0	
edit	12	2022-09-30 23:48:00	23.83	0.52	1019.27	30.01	1.6	45	3.2	45	0	0	13.37	23.83	0	0	
edit	13	2022-09-30 23:47:00	23.78	0.52	1019.27	30.01	1.6	45	3.2	45	0	0	13.32	23.78	0	0	
edit	14	2022-09-30 23:46:00	23.78	0.52	1019.37	30.01	1.6	22.5	1.6	23	0	0	13.32	23.78	0	0	
edit	15	2022-09-30 23:45:00	23.78	0.52	1019.34	30.01	1.6	22.5	3.2	23	0	0	13.32	23.78	0	0	
edit	16	2022-09-30 23:44:00	23.72	0.53	1019.4	30.01	0	1.6	0	0	0	13.56	23.72	0	0	NULL	
edit	17	2022-09-30 23:43:00	23.72	0.53	1019.47	30.02	1.6	0	1.6	0	0	0	13.56	23.72	0	0	
edit	18	2022-09-30 23:42:00	23.72	0.53	1019.4	30.01	1.6	0	1.6	0	0	0	13.56	23.72	0	0	
edit	19	2022-09-30 23:41:00	23.72	0.53	1019.37	30.01	0	1.6	0	0	0	13.56	23.72	0	0	NULL	

Εικόνα 3.5: Ο πίνακας weather_data με τις αποθηκευμένες μετρήσεις (106.138 γραμμές).

2. **Επίπεδο βάσης:** ο UNIQUE constraint στη στήλη timestamp λειτουργεί ως δίχτυ ασφαλείας — ακόμη και αν το επίπεδο εφαρμογής αποτύχει, η βάση θα απορρίψει τις διπλές εγγραφές.

3.3 Κύρια ανάλυση — σενάρια οπτικοποίησης

Παρουσιάζονται τρία υλοποιημένα σενάρια που αναδεικνύουν την αξιοποίηση των δεδομένων.

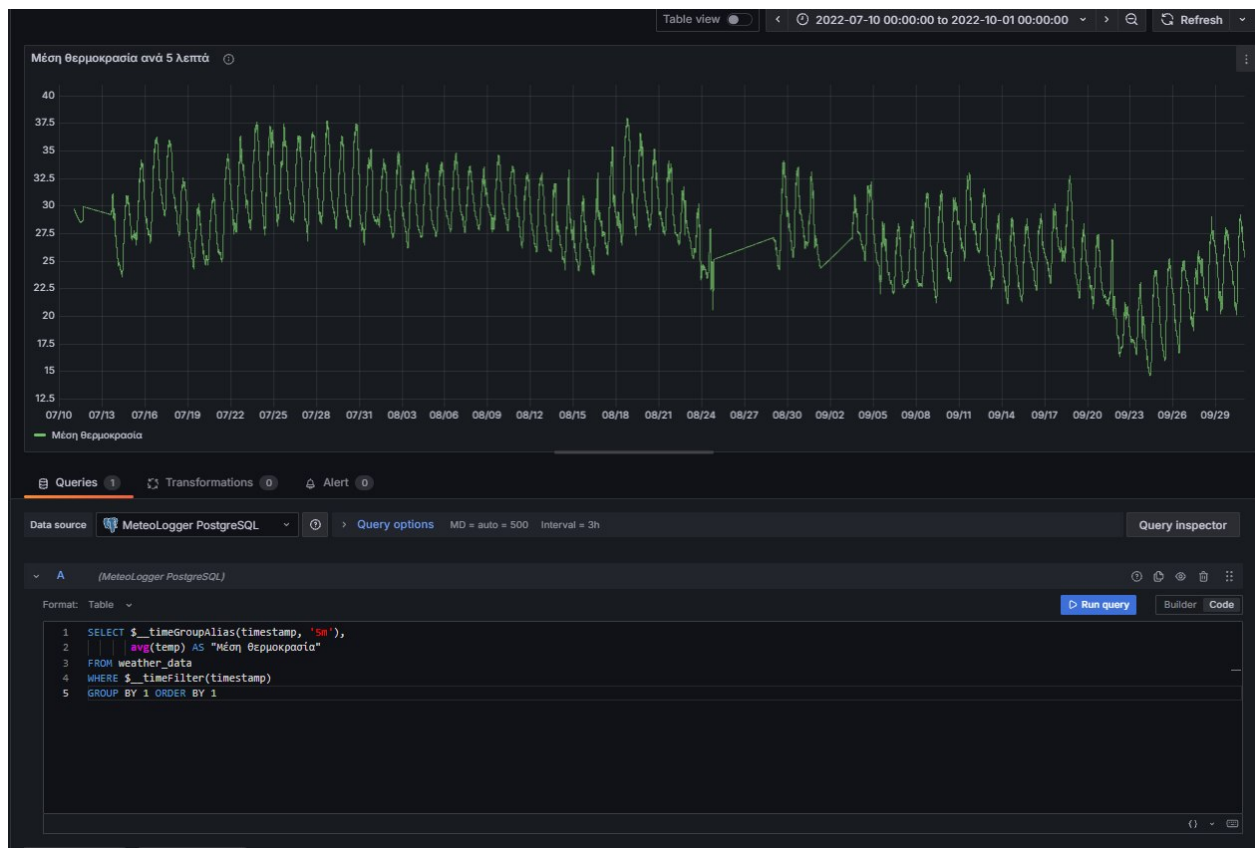
3.3.1 Σενάριο 1 — Μέση θερμοκρασία ανά 5 λεπτά

Επειδή ο σταθμός καταγράφει μία μέτρηση ανά λεπτό, το ερώτημα υπολογίζει τον **μέσο όρο ανά 5 λεπτά**, ώστε να μη σχεδιάζονται χιλιάδες σημεία που επιβαρύνουν το γράφημα χωρίς να προσθέτουν πληροφορία:

```
SELECT
    $__timeGroupAlias(timestamp, '5m'),
    avg(temp) AS "M"
FROM weather_data
WHERE $__timeFilter(timestamp)
GROUP BY 1
ORDER BY 1
```

Στην Εικόνα 8 φαίνεται το ερώτημα μαζί με το γράφημα, και στην Εικόνα 9 το τελικό γράφημα για όλη την περίοδο. Διακρίνονται ευκρινώς τα **ημερήσια μοτίβα** (άνοδος τη μέρα, πτώση τη νύχτα), οι καλοκαιρινές **αιχμές κοντά στους 38 °C** κατά τις μεσημεριανές ώρες του Ιουλίου και Αυγούστου, και η **σταδιακή πτώση** της θερμοκρασίας προς τα τέλη Σεπτεμβρίου.

Η μέθοδος χρονικής ομαδοποίησης (time bucketing) είναι θεμελιώδης τεχνική για την αποδοτική οπτικοποίηση χρονοσειρών υψηλής συχνότητας: αντί να σχεδιάζονται 106.138 σημεία, σχεδιάζονται περίπου 21.228 (μείωση ~80%), χωρίς ουσιαστική απώλεια πληροφορίας για μακροπρόθεσμη εξέταση.



Εικόνα 3.6: Το ερώτημα SQL και το γράφημα που παράγει (Σενάριο 1).



Εικόνα 3.7: Μέση θερμοκρασία ανά 5 λεπτά, Ιούλιος–Σεπτέμβριος 2022.

3.3.2 Σενάριο 2 — Καταγραφή μόνο στην αλλαγή (αναφορά κατ' εξαίρεση)

Το σενάριο αυτό υλοποιεί τη λογική των SCADA γνωστή ως **exception reporting**: κρατά ένα σημείο μόνο όταν η θερμοκρασία αλλάξει κατά έναν ολόκληρο βαθμό. Χρησιμοποιεί τη window function LAG της SQL για να συγκρίνει κάθε τιμή με την αμέσως προηγούμενη:

```
SELECT "time", temp AS "Θ"      (      )"
FROM (
    SELECT timestamp AS "time", temp,
           round(temp::numeric) AS deg,
           lag(round(temp::numeric)) OVER (ORDER BY timestamp) AS prev_deg
    FROM weather_data
    WHERE $__timeFilter(timestamp)
) q
WHERE prev_deg IS NULL OR deg <> prev_deg
ORDER BY "time"
```

Το αποτέλεσμα είναι εντυπωσιακή **μείωση όγκου**: από 106.138 μετρήσεις διατηρούνται μόνο **3.858** σημεία, δηλαδή μείωση **~96,4%**, χωρίς να χάνεται η ουσιαστική εικόνα της διακύμανσης. Αυτό σημαίνει ότι για μια εβδομαδιαία ή μηνιαία εξέταση, ο χρήστης βλέπει γράφημα που φορτώνει σχεδόν ακαριαία.

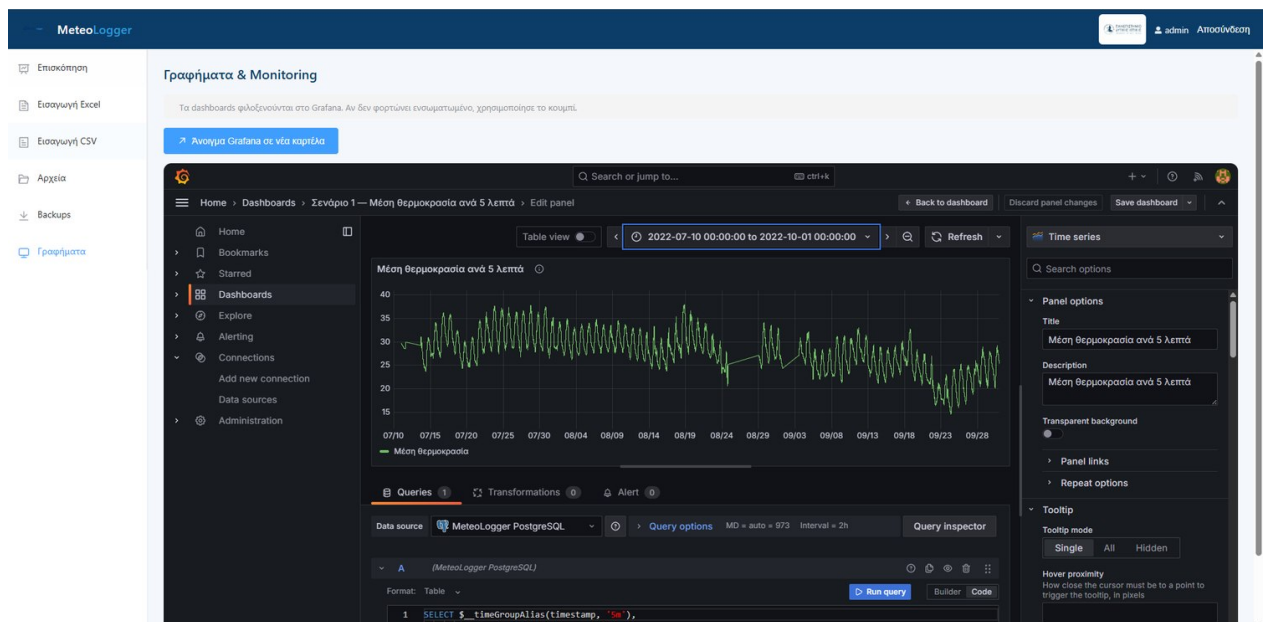
3.3.3 Σενάριο 3 — Ημερήσια μέγιστα ηλιακής ακτινοβολίας

Ένα τρίτο σενάριο αναδεικνύει το ημερήσιο μέγιστο της ηλιακής ακτινοβολίας ανά ημέρα, χρήσιμο για την εκτίμηση της ενεργειακής απόδοσης φωτοβολταϊκών συστημάτων:

```
SELECT
    date_trunc('day', timestamp) AS "H ",
    max(radiation) AS "M"      (W/m²)"
FROM weather_data
WHERE $__timeFilter(timestamp) AND radiation IS NOT NULL
GROUP BY 1
ORDER BY 1
```

Το ερώτημα αυτό παράγει έναν πίνακα τιμών (μία ανά ημέρα) που μπορεί να χρησιμοποιηθεί ως input για ενεργειακούς υπολογισμούς.

Τέλος, τα γραφήματα ενσωματώνονται και μέσα στο διαχειριστικό μέσω iframe (Εικόνα 10), ώστε ο χρήστης να μην χρειάζεται να πλοηγηθεί ξεχωριστά στο Grafana.



Εικόνα 3.8: Ενσωμάτωση των γραφημάτων του Grafana στο διαχειριστικό.

Κεφάλαιο 4

Συζήτηση αποτελεσμάτων

4.1 Επαλήθευση ερευνητικών υποθέσεων

Τα αποτελέσματα υποστηρίζουν και τις τρεις ερευνητικές υποθέσεις με σαφή τρόπο.

Ως προς την **Y1**, ολοκληρώθηκε πλήρως λειτουργική πλατφόρμα αποκλειστικά με εργαλεία ανοικτού λογισμικού (Python, FastAPI, PostgreSQL, Grafana, Vue, Docker, Nginx). Η επιτυχής διαδρομή από την εισαγωγή αρχείου Excel έως το τελικό διαδραστικό γράφημα αποδεικνύει ότι ο συνδυασμός αυτών των τεχνολογιών παράγει λειτουργικό, αξιόπιστο σύστημα, απορρίπτοντας τη μηδενική υπόθεση H0-1.

Ως προς την **Y2**, η αποθήκευση του ίδιου αρχείου δεύτερη φορά δεν πρόσθεσε καμία εγγραφή· ο αριθμός γραμμών στον πίνακα παρέμεινε σταθερός. Η υλοποίηση σε δύο επίπεδα (εφαρμογή + βάση) εξασφαλίζει ισχυρή προστασία έναντι διπλοεγγραφών, απορρίπτοντας την H0-2.

Ως προς την **Y3**, και οι δύο τεχνικές έδειξαν δραστική μείωση: το time bucketing ανά 5 λεπτά μείωσε τα σημεία κατά ~80%, ενώ το exception reporting κατά ~96,4%. Και στις δύο περιπτώσεις η οπτική εικόνα της διακύμανσης παρέμεινε ουσιαστικά αμετάβλητη, επιβεβαιώνοντας ότι οι τεχνικές αυτές βελτιώνουν σημαντικά την απόδοση της οπτικοποίησης, απορρίπτοντας την H0-3.

4.2 Σύγκριση με εμπορικές λύσεις

Σε σύγκριση με εμπορικές λύσεις όπως το WeatherLink της DAVIS, η προτεινόμενη πλατφόρμα προσφέρει **πλήρη πρόσβαση στα ωμά δεδομένα** μέσω SQL, **ελευθερία επέκτασης** (προσθήκη νέων πηγών, νέων ερωτημάτων, νέων οπτικοποιήσεων) και **μηδενικό κόστος αδειών**. Ο πίνακας 2 συνοψίζει τη σύγκριση.

Πίνακας 4.1: Σύγκριση MeteoLogger με εμπορική λύση (WeatherLink).

Κριτήριο	MeteoLogger	WeatherLink
Κόστος λογισμικού	Μηδέν	Συνδρομή
Πρόσβαση στα ωμά δεδομένα	Πλήρης (SQL)	Περιορισμένη
Προσαρμοσμένα γραφήματα	Ναι (SQL)	Προεπιλεγμένα
Επεκτασιμότητα	Υψηλή	Χαμηλή
Εγκατάσταση	Docker (αυτόματη)	Εμπορικό installer
Αντίγραφα ασφαλείας	Αυτόματα (καθημερινά)	Χειροκίνητα

4.3 Σύγκριση με εναλλακτικές ανοικτού κώδικα

Η επιλογή PostgreSQL + Grafana ταιριάζει καλύτερα στο σενάριο **ιστορικών δεδομένων από αρχεία** σε σχέση με τις εναλλακτικές:

- **InfluxDB:** Κατάλληλη για streaming δεδομένα σε πραγματικό χρόνο. Εισάγει εξειδικευμένη γλώσσα (Flux) και το μοντέλο δεδομένων της δεν ταιριάζει φυσικά με σχεσιακή δομή (συσκευές, χρήστες, μεταδεδομένα).
- **Prometheus:** Προορίζεται αποκλειστικά για παρακολούθηση υποδομών (scrapping) σε πραγματικό χρόνο. Δεν υποστηρίζει εισαγωγή ιστορικών αρχείων.
- **TimescaleDB:** Ενδιαφέρουσα επιλογή ως επέκταση της PostgreSQL, αλλά εισάγει πολυπλοκότητα (hypertables, continuous aggregates) που δεν δικαιολογείται για τον παρόντα όγκο δεδομένων.

4.4 Περιορισμοί και απειλές εσωτερικής εγκυρότητας

Ένας σημαντικός περιορισμός είναι ότι το δείγμα καλύπτει μόνο **καλοκαιρινή περίοδο** (Ιούλιος–Σεπτέμβριος). Αυτό σημαίνει ότι: (α) όρια που αφορούν ψύχος ή θυελλώδη άνεμο δεν ελέγχθηκαν στην πράξη, και (β) τα στατιστικά της περιγραφικής ανάλυσης δεν αντιπροσωπεύουν ολόκληρο το έτος. Ωστόσο, αυτό δεν αναιρεί τη λειτουργία των μηχανισμών — μόνο περιορίζει την εμβέλεια των συμπερασμάτων στη συγκεκριμένη περίοδο.

Δεύτερος περιορισμός είναι ότι η δοκιμή έγινε σε **τοπικό δίκτυο**, χωρίς να ελεγχθεί η συμπεριφορά υπό πραγματικό φόρτο πολλαπλών ταυτόχρονων χρηστών ή με δεδομένα ετών (εκατομμύρια γραμμές). Για παραγωγική χρήση σε μεγαλύτερη κλίμακα θα χρειαστούν βελτιστοποιήσεις στα ευρετήρια ή, ενδεχομένως, επέκταση με TimescaleDB.

Κεφάλαιο 5

Συμπεράσματα και προτάσεις

5.1 Συμπεράσματα

Υλοποιήθηκε μια ολοκληρωμένη, ανοικτή πλατφόρμα για τη συλλογή, αποθήκευση και παρακολούθηση χρονοσειρών μετρήσεων. Επιτεύχθηκαν όλοι οι αρχικοί στόχοι:

- **Εύκολη εισαγωγή** από αρχεία Excel/CSV με αυτόματο εντοπισμό κεφαλίδων και χρονικής σφραγίδας.
- **Αξιόπιστη αποθήκευση** με μοναδικό κλειδί τη χρονική στιγμή, που αποτρέπει τις διπλοεγγραφές σε δύο επίπεδα.
- **Ποιοτικός έλεγχος** με φυσικά όρια και διασταύρωση τιμών, που μηδενίζει μη έγκυρες τιμές χωρίς απώλεια ολόκληρης της γραμμής.
- **Ισχυρή οπτικοποίηση** μέσω Grafana με SQL ερωτήματα, που επιτρέπει χρονικές ομαδοποιήσεις και exception reporting.
- **Εύκολη, αναπαραγώγιμη εγκατάσταση** μέσω Docker Compose με μία εντολή.
- **Ασφαλής πρόσβαση** μέσω JWT tokens και αυτόματα αντίγραφα ασφαλείας.

Η υλοποίηση αποδεικνύει ότι ο συνδυασμός ανοικτών τεχνολογιών (Python/FastAPI, PostgreSQL, Grafana, Vue, Docker) μπορεί να παράγει επαγγελματικό αποτέλεσμα χωρίς κόστος αδειών, κατάλληλο για ακαδημαϊκά, ερευνητικά και μικρομεσαία βιομηχανικά περιβάλλοντα.

5.2 Δυσκολίες που αντιμετωπίστηκαν

Κατά την υλοποίηση προέκυψαν αρκετές τεχνικές δυσκολίες που αντιμετωπίστηκαν επιτυχώς:

- **Ποιότητα δεδομένων:** μετατοπισμένες στήλες και φυσικά αδύνατες τιμές στα αρχεία WeatherLink, που αντιμετωπίστηκαν με τον έλεγχο φυσικών ορίων ανά μέγεθος.
- **Ανάγνωση αρχείων με άσχετες γραμμές:** τα αρχεία WeatherLink έχουν γραμμές metadata στην αρχή· λύθηκε με αλγόριθμο αυτόματου εντοπισμού κεφαλίδων.
- **Χρονική σφραγίδα:** σωστός συνδυασμός ξεχωριστών στηλών ημερομηνίας και ώρας με αποφυγή ασαφειών ζώνης ώρας.
- **Ρύθμιση δικτύου containers:** ορισμός εσωτερικού δικτύου Docker, ονοματισμένη πρόσβαση μέσω virtual hosts και σωστή ρύθμιση reverse proxy Nginx.

- **Ενσωμάτωση Grafana:** αποφυγή CORS σφαλμάτων κατά την ενσωμάτωση γραφημάτων μέσω iframe στο διαχειριστικό.

5.3 Προτάσεις για μελλοντική επέκταση

Βάσει της εμπειρίας της υλοποίησης και των περιορισμών που εντοπίστηκαν, προτείνονται οι ακόλουθες επεκτάσεις:

- **Άμεση επικοινωνία με τα όργανα** (Modbus TCP/RTU, IEC 60870-5-104, IEC 61850) αντί για χειροκίνητο ανέβασμα αρχείων, επιτρέποντας συλλογή δεδομένων σε πραγματικό χρόνο.
- **Υποστήριξη περισσότερων οργάνων** (καταγραφικά ηλεκτρικών μεγεθών, αισθητήρες θερμοροής) με πίνακες συσκευών/μεταδεδομένων στη βάση.
- **Διαχείριση χρηστών** με ρόλους και δικαιώματα (π.χ. διαχειριστής, αναλυτής, επισκέπτης).
- **Ειδοποιήσεις (alerting)** σε υπερβάσεις ορίων, μέσω του ενσωματωμένου μηχανισμού Grafana Alerting.
- **Παρακολούθηση της υποδομής** με Prometheus (υγεία containers, χρήση πόρων) και ενσωμάτωση των μετρικών υποδομής στο ίδιο Grafana instance.
- **Αυτόματος καθαρισμός παλαιών δεδομένων** με πολιτική διατήρησης (retention policy) για εξοικονόμηση χώρου.
- **TimescaleDB** για βελτίωση επιδόσεων εφόσον ο όγκος δεδομένων αυξηθεί σε δεκάδες εκατομμύρια γραμμές.
- **API για εξωτερική πρόσβαση** (REST endpoints για λήψη δεδομένων σε JSON ή CSV), επιτρέποντας ενσωμάτωση με άλλα συστήματα.

Βιβλιογραφία

- [1] FastAPI, «FastAPI documentation», <https://fastapi.tiangolo.com>
- [2] The PostgreSQL Global Development Group, «PostgreSQL Documentation», <https://www.postgresql.org/docs/>
- [3] Grafana Labs, «Grafana documentation», <https://grafana.com/docs/>
- [4] Docker Inc., «Docker documentation», <https://docs.docker.com>
- [5] Vue.js, «Vue.js documentation», <https://vuejs.org>
- [6] pandas, «pandas documentation», <https://pandas.pydata.org/docs/>
- [7] DAVIS Instruments, «Weather Stations», <https://www.davisinstruments.com>
- [8] SQLAlchemy, «SQLAlchemy Documentation», <https://docs.sqlalchemy.org>
- [9] Nginx, «Nginx Documentation», <https://nginx.org/en/docs/>
- [10] Python Software Foundation, «Python 3.11 Documentation», <https://docs.python.org/3.11/>

Παράρτημα Α — Πρόσθετα ερωτήματα

Πέρα από τα τρία σενάρια της κύριας ανάλυσης, καταγράφονται εδώ ενδεικτικά πρόσθετα ερωτήματα που υλοποιήθηκαν για τη δοκιμή του συστήματος.

Ανίχνευση ακραίων θερμοκρασιών (>35 °C):

```
SELECT timestamp AS "time", temp AS "Θ"
FROM weather_data
WHERE $__timeFilter(timestamp) AND temp > 35
ORDER BY timestamp
```

Έντονη βροχόπτωση (ρυθμός > 20 mm/h):

```
SELECT timestamp AS "time", rainrate AS "Ρ"
FROM weather_data
WHERE $__timeFilter(timestamp) AND rainrate > 20
ORDER BY timestamp
```

Ημερήσια στατιστικά θερμοκρασίας:

```
SELECT
  date_trunc('day', timestamp) AS "H",
  min(temp) AS "E (°C)",
  avg(temp) AS "M (°C)",
  max(temp) AS "M (°C)"
FROM weather_data
WHERE $__timeFilter(timestamp)
GROUP BY 1
ORDER BY 1
```

Ώρες υψηλής UV ακτινοβολίας (UV > 8):

```
SELECT timestamp AS "time", uv AS "Δ UV"
FROM weather_data
WHERE $__timeFilter(timestamp) AND uv > 8
ORDER BY timestamp
```

Παράρτημα Β — Ενδεικτικά τμήματα κώδικα

5.4 B.1 Ορισμός του πίνακα μετρήσεων (SQLAlchemy Model)

```
class WeatherData(Base):
    __tablename__ = "weather_data"
    id = Column(Integer, primary_key=True, index=True)
    timestamp = Column(DateTime, unique=True, nullable=False, index=True)
    temp = Column(Float); humidity = Column(Float)
    barometer = Column(Float); pressure = Column(Float)
    windspeed = Column(Float); winddirection = Column(Float)
    windgust = Column(Float); windgustdir = Column(Float)
    rain = Column(Float); rainrate = Column(Float)
    dewpoint = Column(Float); windchill = Column(Float)
    radiation = Column(Float); uv = Column(Float)
```

5.5 B.2 Αυτόματος εντοπισμός κεφαλίδων

```
def find_header_row(filepath: str) -> int:
    """E . """
    expected = {"Date", "Time", "Temp", "Humidity"}
    df_raw = pd.read_excel(filepath, header=None, nrows=20)
    for i, row in df_raw.iterrows():
        if expected.issubset(set(str(v) for v in row)):
            return i
    raise ValueError("Δ ")
```

5.6 B.3 Εισαγωγή αρχείου χωρίς διπλοεγγραφές (απλοποιημένα)

```
@router.post("/import/excel")
async def import_excel(file: UploadFile, db: Session = Depends(get_db)):
    content = await file.read()
    stored = save_to_disk(content, file.filename) # (host)
```

```

header_row = find_header_row(stored)
df = pd.read_excel(stored, header=header_row) #
df = build_timestamp(df) #
df = clean_numeric(df) #
df = validate_ranges(df) #
existing = set(get_all_timestamps(db))
df_new = df[~df["timestamp"].isin(existing)] #
insert_rows(db, df_new)
return {
    "rows_inserted": len(df_new),
    "rows_skipped": len(df) - len(df_new),
    "filename": file.filename
}

```

5.7 B.4 Έλεγχος φυσικών ορίων

```

BOUNDS = {
    "temp": (-40, 60),
    "humidity": (0, 100),
    "barometer": (800, 1100),
    "windspeed": (0, 200),
    "dewpoint": (-40, 40),
    "windchill": (-60, 60),
    "radiation": (0, 1500),
    "uv": (0, 20),
}

```

```

def validate_ranges(df: pd.DataFrame) -> pd.DataFrame:
    for col, (lo, hi) in BOUNDS.items():
        if col in df.columns:
            mask = df[col].between(lo, hi)
            df[col] = df[col].where(mask) # -> NULL (NaN)
    # ϕ ; <=
    if "dewpoint" in df.columns and "temp" in df.columns:
        bad = df["temp"].notna() & (df["dewpoint"] > df["temp"] + 0.5)
        df.loc[bad, "dewpoint"] = None
    return df

```

5.8 B.5 Απόσπασμα ρύθμισης του Docker Compose

```

version: "3.9"
services:
    db:

```

```

    image: postgres:15
    environment:
      POSTGRES_DB: measurelog
      POSTGRES_USER: meteologger
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - db_data:/var/lib/postgresql/data

api:
  build: ./backend
  depends_on: [db]
  environment:
    DATABASE_URL: postgresql://meteologger:${DB_PASSWORD}@db:5432/measurelog

frontend:
  build: ./frontend

grafana:
  image: grafana/grafana:11.4.0
  volumes:
    - grafana_data:/var/lib/grafana

adminer:
  image: adminer

proxy:
  image: nginx:1.27-alpine
  ports: ["80:80"]
  volumes:
    - ./nginx.conf:/etc/nginx/nginx.conf:ro

backup:
  build: ./backup
  environment:
    POSTGRES_HOST: db
    POSTGRES_DB: measurelog
    KEEP_DAYS: 7

volumes:
  db_data:
  grafana_data:

```